was, the operation is not performed, and an exception is thrown. The check is based on a numeric field that starts counting from 0 and automatically increases at every update operation. You can mark such a field with the `@Version` annotation (from the `org.springframework.data.annotation` package).

When the `@Id` field is `null` and the `@Version` field is `0`, Spring Data JDBC assumes it's a new object. Consequently, it relies on the database to generate an identifier when inserting the new row in the table. When values are provided, it expects to find the object in the database already and to update it.

Let's go ahead and add two new fields to the `Book` record for the identifier and the version number. Since both fields are populated and handled by Spring Data JDBC under the hood, using the all-args constructor might be too verbose for situations such as generating test data. For convenience, let's add a static factory method to the `Book` record for building an object by passing only the business fields.

Listing 5.4 Defining identifier and version for `Book` objects

```
package com.polarbookshop.catalogservice.domain;

public record Book (

  @Id                                          ❶
  Long id,

  @NotBlank(message = "The book ISBN must be defined.")
  @Pattern(
    regexp = "^([0-9]{10}|[0-9]{13})$",
    message = "The ISBN format must be valid."
  )
  String isbn,

  @NotBlank(message = "The book title must be defined.")
  String title,

  @NotBlank(message = "The book author must be defined.")
```